

PML-AI Body of Knowledge

The reference for the PCI Project Management Leader – AI
Leadership · delivery systems · governance · the governed use
of AI

FIRST EDITION — PHASE 1 PROTOTYPE (NOT FOR RELEASE)

PROJECT CONTROLS INSTITUTE GLOBAL

Finance intelligently. Control predictively. Deliver successfully.

AI proposes; the professional verifies, decides and remains accountable.

— Prototype notice

This document is a **Phase 1 production prototype** of one domain, built to validate the book's pattern, calculation-verification harness, figures and typesetting pipeline before scaled authorship. It is not a released PCI publication and carries no entitlement, syllabus or examination status. The released edition will carry the full copyright, disclaimer and trademark notices of the PCI book family; all examples here are original and fictional, all calculations pass the golden-answer suite ([verify_formulas.py](#)), and no standard's text is reproduced.

Contents

Planning, Scheduling and Delivery Flow (quantitative flagship)	5
Why this domain exists	5
Knowledge Area 6.1 Planning levels and logic networks.....	6
Knowledge Area 6.2 The critical path and float	9
Knowledge Area 6.3 Resources, constraints, milestones and rolling wave	12
Knowledge Area 6.4 Delivery flow: predictive, agile and hybrid; recovery and forecasting.....	16
Advanced topics — Domain 6	20
Case study — Domain 6: recovering Auriga (utilities / technology)	20
Case study B — Domain 6: the milestone that was typed, not computed (public programme)	21
Executive perspective — Domain 6	21
Calculation exercises — Domain 6.....	22
Practitioner's toolkit — Domain 6	22
Exam preparation — Domain 6	23
Domain 6 summary	23

01

PART TWO

Delivering the work

Domains 5-10 — scope, planning and scheduling, cost and commercial awareness, risk, quality and procurement: the delivery disciplines a leader integrates.



06

DOMAIN 6

Planning, Scheduling and Delivery Flow (quantitative flagship)

Group: Delivering the work (Domain 6 of 6 in Part Two). **Target:** ~76 pages. **Binds to:** the PCI Book Pattern Specification and the shared registries (<docs/books/registries/>). This domain is the home of the schedule symbols — **ES**, **EF**, **LS**, **LF**, **TF** (total float), **FF** (free float) — restated by every domain that touches time. British English; USD (+SAR where useful, indicative **USD 1 ≈ SAR 3.75**).

— Why this domain exists

A project leader's authority rests on one repeated act: making a credible statement about the future — when the work will finish, what could move that date, and what it would cost to move it back. This domain builds the machinery behind that credibility. It starts where planning actually starts — with levels of plan and the logic that binds work together (KA 6.1); builds the critical path method in full, forward and backward passes, and both kinds of float (KA 6.2); adds the realities the textbook network ignores — resources, constraints, milestones and rolling-wave elaboration (KA 6.3); and finishes with delivery flow across predictive, agile and hybrid worlds: recovery, compression economics, and forecasting under uncertainty (KA 6.4). Cost joins schedule in Domain 7 (earned value); risk quantification deepens in Domain 8. A leader who cannot read a network diagram is hostage to whoever can; this domain removes the hostage-taking.

Learning objectives. After this domain a candidate can: choose the right planning level for an audience and a decision; build a logic network with correct dependency types; run forward and backward passes and derive total and free float; identify the critical path and near-critical paths and explain why float is not spare time; plan resources against the schedule and level them deliberately; apply rolling-wave elaboration honestly; select and price schedule compression; distinguish predictive, agile and hybrid scheduling and govern each; and use three-point estimates and scenario analysis to forecast completion with stated uncertainty — with any AI-generated schedule subject to the family's verification rule.

The master worked project. One project runs through this domain and returns in Domains 7 and 8: **Project Auriga**, a control-systems upgrade for a regional utility, planned in weeks:

ID	Activity	Duration (wk)	Predecessors
A	Mobilise	2	—
B	Detailed design	6	A
C	Procure control hardware	8	B
D	Civil and cabling works	7	B
E	Installation	5	C, D
F	Testing and commissioning	4	E
G	Operator training and handover	2	D

Every calculation below uses this network.

KNOWLEDGE AREA 6.1

Planning levels and logic networks

Topics: 6.1.1 levels of plan · 6.1.2 dependencies and logic · 6.1.3 network rules and quality.

6.1.1 Levels of plan

The principle. One schedule cannot serve a board, a site supervisor and a lender at once. Mature delivery organisations maintain a **hierarchy of schedules**, each derived from the one below, each honest with the one above:

Level	Name	Audience & decision	Typical granularity
L1	Executive summary	Board, sponsor — go/no-go, milestones	10–30 bars
L2	Management schedule	Steering, PMO — stage gates, interfaces	50–300 activities
L3	Control schedule	Project team — the CPM network; progress and float live here	300–5,000 activities
L4	Execution/lookahead	Site and squads — 2–6 week detail	daily/shift detail

The **control schedule (L3)** is where this domain's machinery operates: it carries the logic, the floats and the critical path, and it is the version under change control (Domain 4, KA 4.3). An L1 milestone that cannot be traced to L3 logic is decoration, and steering committees should treat it as such (Domain 3, KA 3.2).

6.1.2 Dependencies and logic

Definitions. Activities link through four dependency types — finish-to-start (**FS**, the default: design finishes, then procurement starts), start-to-start (**SS**: cabling can start once civils have started, often with a lag), finish-to-finish (**FF**: commissioning documentation finishes when commissioning finishes), and the rare start-to-finish (**SF**). A **lag** delays a successor ($FS+2$ = start two weeks after the predecessor finishes); a **lead** (negative lag) overlaps it. Logic expresses physics and choice — what

must follow, and what the team has chosen to sequence — and the two should be distinguishable in the schedule's notes, because only chosen logic can be re-chosen when recovery demands it (KA 6.4).

Common pitfall — the lag that hides work. A 3-week lag labelled "curing" is physics; a 3-week lag labelled nothing is often an activity somebody didn't model — unresourced, uncosted, invisible to progress measurement. Audit rule: every lag has a stated reason, and long lags (> 10 % of project duration) are activities in disguise.

WORKED EXAMPLE

Worked example 6.1.2 — re-choosing a dependency.

1. **Setup.** Survey work P (4 weeks) precedes report drafting Q (6 weeks), currently linked FS+2 (two weeks of data cleaning after all surveying ends). The team proposes drafting in parallel as survey sections complete: SS+1. Compute Q's dates both ways (P starts week 0).
2. **Formula.** FS+lag: $ES(Q) = EF(P) + \text{lag}$. SS+lag: $ES(Q) = ES(P) + \text{lag}$.
3. **Substitution.** FS+2: $ES(Q) = 4 + 2 = 6$, $EF(Q) = 12$. SS+1: $ES(Q) = 0 + 1 = 1$, $EF(Q) = 7$.
4. **Result.** The chain finishes week **12** under FS+2 and week **7** under SS+1 — five weeks earlier from one logic choice.
5. **Interpretation.** Nothing was crashed and nobody works faster: the saving comes from overlapping, and its price is rework risk if early sections change after later surveying. This is why chosen logic must be visibly chosen (not fossilised as physics) — it is the cheapest recovery currency the project owns (KA 6.4.2), and spending it is a risk decision, not a scheduling trick.

6.1.3 Network rules and quality

A network the passes can trust obeys checkable rules: every activity except the first has a predecessor and except the last a successor (**no dangles**); no loops; durations estimated by the owning discipline, not the scheduler; no date constraints doing logic's job (a "must start on" pin that overrides logic converts the schedule from a model into a poster); and logic density in a healthy range — too few links and float is fiction, too many and nothing can move. These rules are the schedule-quality gate a leader can run without scheduling software: ask for the dangle count, the constraint count and the reasons for both.

AI in this KA

Scheduling tools now draft networks from scope statements and historical libraries. Useful — the draft logic is often 80 % right — and dangerous in a specific way: generated logic looks authoritative while encoding assumptions nobody made. The governed workflow: AI proposes the network; the discipline leads walk every link of their scope ("does B truly need all of A finished?"); the scheduler runs the quality rules above; and the leader signs the logic as a decision record (Domain 3, KA 3.3.4).

AI proposes; the professional verifies, decides and remains accountable.

■ KEY TERMS — KA 6.1

Term	Meaning
Schedule hierarchy (L1-L4)	Nested plans for different audiences, derived from one logic.
Control schedule	The L3 CPM network under change control; home of float and progress.
FS / SS / FF / SF	The four dependency types.
Lag / lead	Imposed delay / overlap on a dependency.
Dangle	An activity missing a predecessor or successor; a network defect.
Date constraint	A pinned date overriding logic; each one weakens the model.

■ SAMPLE MCQS — KA 6.1

MCQ 6.1-A [6.1.2 · Application] Cabling may begin one week after civil works begin. The correct dependency is: - A. FS+1 - B. SS+1 - C. FF+1 - D. FS-1

Rationale: The condition binds the two starts with a one-week lag: start-to-start plus one. A would wait for all civils to finish; C binds the finishes; D (a lead on FS) overlaps the finish, which is not what was stated.

MCQ 6.1-B [6.1.3 · Analysis] A schedule shows 14 "must finish on" constraints, all on milestones reported to the board. The most accurate reading is: - A. the schedule is well-governed, because board dates are protected - B. the milestones will be met, because the tool will honour the constraints - C. the schedule may no longer model reality — pinned dates can mask logic-driven slippage that float analysis would otherwise reveal - D. constraints are neutral scheduling hygiene with no analytical effect

Rationale: Pins override logic: the network can be slipping while pinned milestones hold still, hiding negative float until it is unrecoverable. A and B mistake suppression for control; D is false — every pin degrades the model's predictive value.

MCQ 6.1-C [6.1.2 · Application] Survey P (4 wk) feeds report Q (6 wk). Under FS+2 the chain completes week 12. Re-linked SS+1, it completes week: - A. 7 - B. 11 - C. 12 - D. 5

Rationale: $ES(Q) = ES(P) + 1 = 1$, $EF(Q) = 7$. B subtracts one lag week from 12; C assumes logic changes nothing; D forgets Q's own duration must still run.

MCQ 6.1-D [6.1.3 · Recall] A "dangle" in a schedule network is: - A. an activity with negative float - B. an activity missing a predecessor or successor link - C. a milestone with zero duration - D. a lag longer than its predecessor

Rationale: Dangles are unbound activity ends — the passes cannot constrain them, so their dates and float are unreliable. A describes constraint-driven lateness; C describes every milestone; D is unusual but legal when justified.

■ SELF-CHECK — KA 6.1

1. Why must chosen logic be distinguishable from physical logic? — Because recovery (KA 6.4) works by re-choosing choices; physics cannot be renegotiated.
2. What three numbers give a fast schedule-quality read? — Dangle count, constraint count, unexplained-lag count — each should be at or near zero, with reasons for the rest.

KNOWLEDGE AREA 6.2

The critical path and float

Topics: 6.2.1 the forward and backward passes · 6.2.2 total and free float · 6.2.3 reading the critical path.

6.2.1 The forward and backward passes

Definitions. For each activity: **ES/EF** — earliest start/finish (forward pass, left to right: **ES** = latest **EF** of predecessors; **EF** = **ES** + duration); **LS/LF** — latest start/finish that do not delay the project (backward pass, right to left: **LF** = earliest **LS** of successors; **LS** = **LF** - duration). The project duration is the largest **EF** in the network.

Worked example 6.2.1 — Auriga, both passes. The passes are inherently tabular; the labelled table replaces the five-step skeleton, and the interpretation follows (pattern rule).

Act	Dur	ES	EF	LS	LF	TF	FF
A	2	0	2	0	2	0	0
B	6	2	8	2	8	0	0
C	8	8	16	8	16	0	0
D	7	8	15	9	16	1	0
E	5	16	21	16	21	0	0
F	4	21	25	21	25	0	0
G	2	15	17	23	25	8	8

Interpretation. Project duration **25 weeks**; the **critical path is A-B-C-E-F** ($2+6+8+5+4 = 25$), every activity on it with zero float. D can slip one week without moving the end date; G can slip eight. The passes turn a drawing into an instrument: every schedule statement a leader makes in this domain — "we finish in week 25", "D matters more than G", "procurement drives the job" — is read directly off this table.

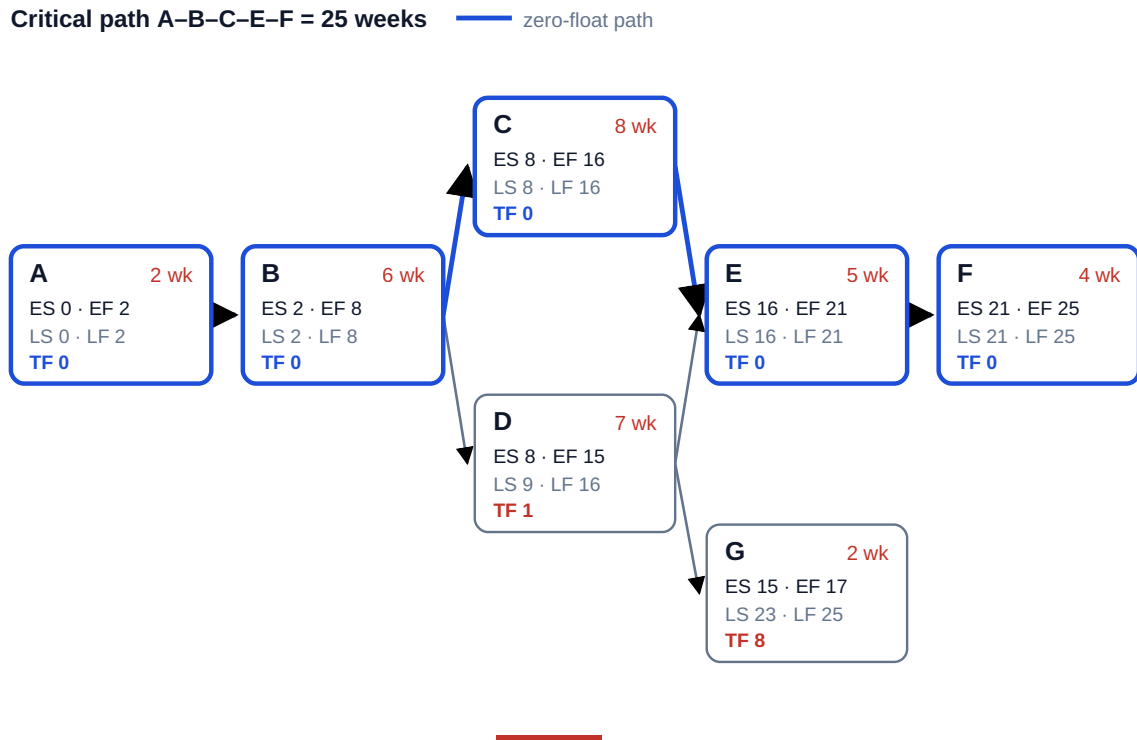


Fig 6.2.1 — The Auriga network with both passes

6.2.2 Total and free float

Definitions.

$TF = LS - ES$ — how far an activity can slip without delaying the project
 $FF = \min(ES \text{ of successors}) - EF$ — how far it can slip without delaying ANY successor

$FF \leq TF$ always. The Auriga table carries the domain's subtlest lesson: **D has $TF = 1$ but $FF = 0$** — D can slip a week without moving the end date, but the moment it slips at all, G's earliest start moves. Total float is a project-level commodity; free float is a courtesy to your successors. A leader who grants a subcontractor "the float" without saying which kind has just given away someone else's schedule.

Float is not spare time. Float belongs to the path, not the activity: consume D's week and every activity sharing that path inherits zero. Contract regimes differ on who owns float (Domain 10, KA 10.3, treats float ownership clauses); the delivery answer is simpler — float is a risk buffer under the leader's governance, spent deliberately or not at all.

Common pitfall — managing only the critical path. D sits one week from critical. A team watching only A-B-C-E-F will discover D's importance the week it becomes too late to matter. Near-critical analysis (all paths within, say, 10 % of project duration) is standing practice; KA 6.4's scenario work builds on it.

6.2.3 Reading the critical path

The critical path is the longest path and therefore the shortest possible duration — both statements are the same fact, and fluency means holding both. Three leader-grade readings of the Auriga table: **(1)** procurement (C) is the single most schedule-driving activity — an expediting conversation is worth more than any amount of site pressure; **(2)** the C/D pair converge on E, so E's start is hostage

to the later of two chains — convergence points are where slippage compounds and where progress review should focus (Domain 8, KA 8.1); **(3)** G's eight weeks of float make it the natural donor of resources in any recovery (KA 6.4).

AI in this KA

CPM arithmetic is deterministic — ideal for machine computation and instant human verification. The two checks that catch nearly every pass error: the critical path's durations must **sum exactly** to the project duration, and every critical activity must show $TF = 0$ with $ES = LS$. An AI-produced schedule that fails either is broken; one that passes both may still have wrong logic — which is why KA 6.1's link-walking discipline precedes trust in any pass, human or machine.

■ KEY TERMS — KA 6.2

Term	Meaning
Forward / backward pass	Left-to-right ES/EF computation; right-to-left LS/LF computation.
ES EF LS LF	Earliest/latest start and finish per activity.
Total float TF	$LS - ES$; slip available without delaying the project.
Free float FF	Slip available without delaying any successor; $FF \leq TF$.
Critical path	The zero-float longest path; the project's shortest possible duration.
Convergence point	A node where paths merge; slippage compounds there.

■ SAMPLE MCQS — KA 6.2

MCQ 6.2-A [6.2.1 · Application] In the Auriga network, activity D (duration 7, predecessor B finishing week 8, successors E and G) has $LS = 9$ and $ES = 8$. Its total float is: - A. 0 - B. 1 - C. 8 - D. 7

Rationale: $TF = LS - ES = 9 - 8 = 1$. A confuses D with the critical activities; C is G's float; D is the activity's duration, not its float.

MCQ 6.2-B [6.2.2 · Analysis] An activity shows $TF = 1$ and $FF = 0$. Delaying it by one week will: - A. delay the project by one week - B. delay nothing, because total float absorbs the slip - C. delay at least one successor's earliest start while leaving the project end date unchanged - D. be impossible — free float can never be below total float

Rationale: Positive TF protects the end date (not A); zero FF means some successor moves immediately (not B). D inverts the invariant — $FF \leq TF$ always.

MCQ 6.2-C [6.2.3 · Application] Auriga's procurement activity C is crashed from 8 weeks to 6. The new project duration is: - A. 23 weeks - B. 24 weeks - C. 25 weeks - D. 21 weeks

Rationale: After one week of crashing, path B-D-E ($8+7+5+4$ via D) becomes co-critical at 24; the second crashed week buys nothing — duration stays 24. A assumes both weeks convert to project weeks; C forgets the crash entirely; D subtracts from the wrong baseline. (The full economics: KA 6.4.)

MCQ 6.2-D [6.2.1 · Application] Auriga's training activity G runs ES 15–EF 17 with $LF = 25$. Its total float is: - A. 8 - B. 10 - C. 2 - D. 0

Rationale: $TF = LF - EF = 25 - 17 = 8$ (equivalently $LS - ES = 23 - 15$). B ignores G's duration ($25 - 15$); C is G's duration; D confuses G with the critical path.

MCQ 6.2-E [6.2.3 · Analysis] A programme's binding path shows $TF = -3$ after a constraint-honest pass. The professional reading is: - A. the software has malfunctioned — float cannot be negative - B. the plan, as constrained, is already three weeks late; the number sizes the recovery problem and must be reported now - C. delete the constraints so the float returns to zero - D. the project will finish three weeks early

Rationale: Negative float is the gap between what logic needs and what constraints allow — information, not error (A) and not good news (D). C is the pin-and-squeeze suppression this domain's Case B dissects; it hides the problem until it is unrecoverable.

■ SELF-CHECK — KA 6.2

1. State both definitions of the critical path and why they are equivalent. — Longest path through the network; shortest possible project duration — the longest chain of unavoidable work is precisely what bounds how soon the whole can finish.
2. Why is $FF \leq TF$ always? — Delaying any successor is one of the ways of delaying the project; the constraint set defining FF is stricter.
3. Auriga slips: D takes 10 weeks, not 7. New duration? — 27 weeks: D finishes week 18, E waits for it ($18 > 16$), E-F add 9 → the critical path re-routes through D (KA 6.4 works the recovery).

KNOWLEDGE AREA 6.3

Resources, constraints, milestones and rolling wave

Topics: 6.3.1 resource planning and levelling · 6.3.2 constraints and milestones · 6.3.3 rolling-wave planning.

6.3.1 Resource planning and levelling

The principle. The passes of KA 6.2 assume infinite resources; reality staffs the network. Loading resources onto the schedule produces a **histogram** — demand per period — and the histogram almost always spikes. Two responses: **resource smoothing** uses float to flatten demand without moving the end date (slip D and G inside their float to pull crews off the peak); **resource levelling** accepts a later end date if the resource cap is hard. The order of operations is a leadership decision in disguise: smoothing spends float — the project's risk buffer — to save money, and someone accountable should price that trade explicitly, not discover it in a progress meeting.

WORKED EXAMPLE**Worked example 6.3.1 — smoothing Auriga's field crews.**

1. **Setup.** Weeks 9-15: civil works D needs 3 crews; early installation staging inside C needs 2; the site cap is 4 crews. Overlap weeks 9-15 demand 5 — one over cap.
2. **Formula.** Smoothing test: can a demanding activity slip within **TF** (and, if successors matter, **FF**) to clear the peak?
3. **Substitution.** D has **TF** = 1, **FF** = 0: slipping D one week clears one peak week but consumes D's entire float and immediately moves G (**FF** = 0). The staging work inside C has discretionary timing across weeks 9-16.
4. **Result.** Re-sequence the staging within C to weeks 14-16 where D's demand tails off: peak demand falls to 4, the cap holds, the end date holds, and **no float is spent**.
5. **Interpretation.** The cheapest capacity is re-sequencing; the second cheapest is float; the expensive ones are overtime, second shifts and extension. A leader asks for the histogram with the options priced, not just the spike.

WORKED EXAMPLE**Worked example 6.3.1b — when the cap is hard: levelling, priced.**

1. **Setup.** Now suppose the site cap is **3 crews** (a permit condition, not a preference). D needs all 3 for weeks 9-15; the 4-week staging package (2 crews) must finish before E can start (week 16). Delay costs USD 45,000 per week (the client's bonus forgone); a second-shift waiver for the staging crew costs USD 20,000 per week. What does the leader do?
2. **Formula.** Compare total cost of each feasible plan: pure levelling (extend) vs paid capacity (second shift) vs any float-funded re-sequence.
3. **Substitution.** Levelling: staging cannot start until D releases crews (week 16); it runs weeks 16-19, E starts week 20 — project 25 → 29, cost $4 \times 45,000 = 180,000$. Second shift: staging runs weeks 12-15 off-shift, E starts on time — cost $4 \times 20,000 = 80,000$. No float-funded option exists: D's **TF** = 1 cannot host a 4-week move.
4. **Result.** **Buy the second shift: USD 80,000** against USD 180,000 of extension — and the end date holds.
5. **Interpretation.** A hard cap converts a scheduling problem into a procurement problem. The histogram's job is to surface that conversion early enough for the cheap option to exist — second shifts, pre-assembly, off-site staging all have lead times of their own. Levelling that silently extends the project is a decision someone should have been asked to make.

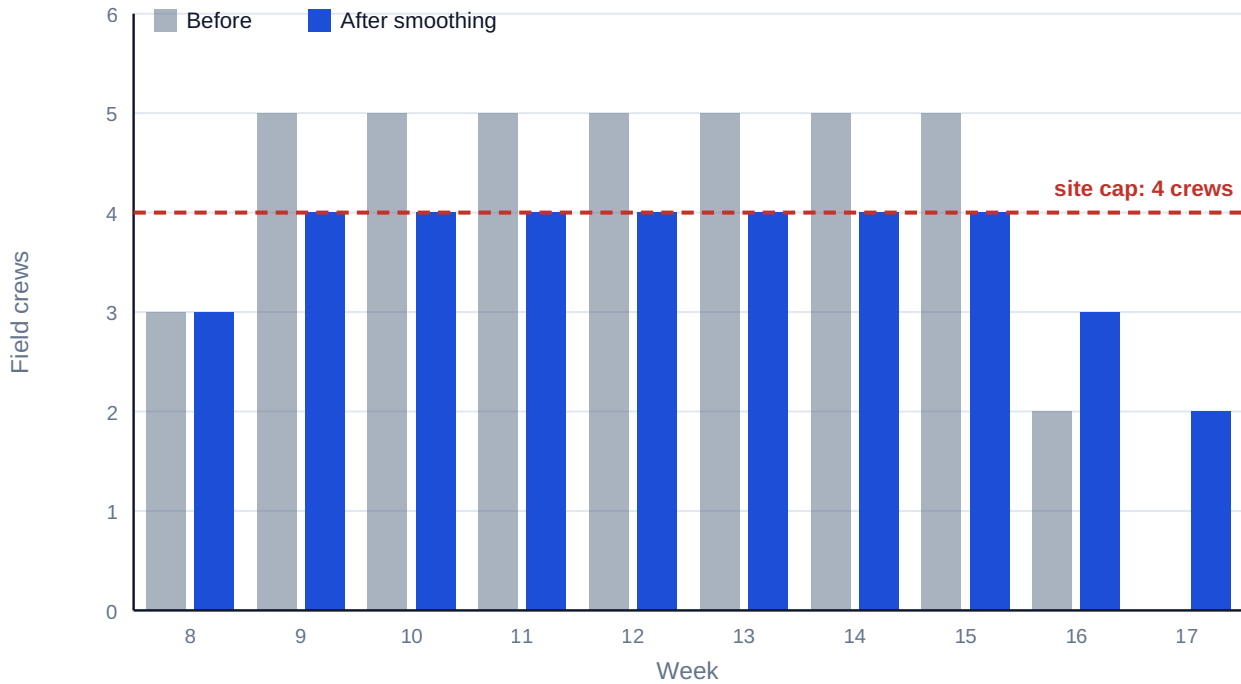


Fig 6.3.1 — Auriga field-crew histogram, before and after smoothing

6.3.2 Constraints and milestones

Constraints are externally imposed dates — an outage window, a regulatory deadline, a seasonal limit. Modelled honestly they are logic (an outage window is a predecessor with a date); modelled lazily they are pins that falsify float (KA 6.1.3). **Milestones** are zero-duration events that carry meaning: contractual obligations (Domain 10), gate decisions (Domain 3), integration points with other projects (Domain 15). A milestone's date is an output of the network, never an input — the moment a milestone is typed rather than computed, the schedule has stopped forecasting and started decorating.

Milestone hygiene: each has an owner, an unambiguous done-definition (Domain 5's acceptance criteria), traceable L3 logic, and — for contractual milestones — a stated float position reported honestly to the counterparty (Domain 11's reporting ethics).

6.3.3 Rolling-wave planning

The principle. Detail decays with distance: estimating month 18's tasks at daily granularity today manufactures precision, not knowledge. **Rolling wave** plans near-term work at execution detail (L4) and far work at planning packages (L3 summary), elaborating each wave as it approaches under change control.

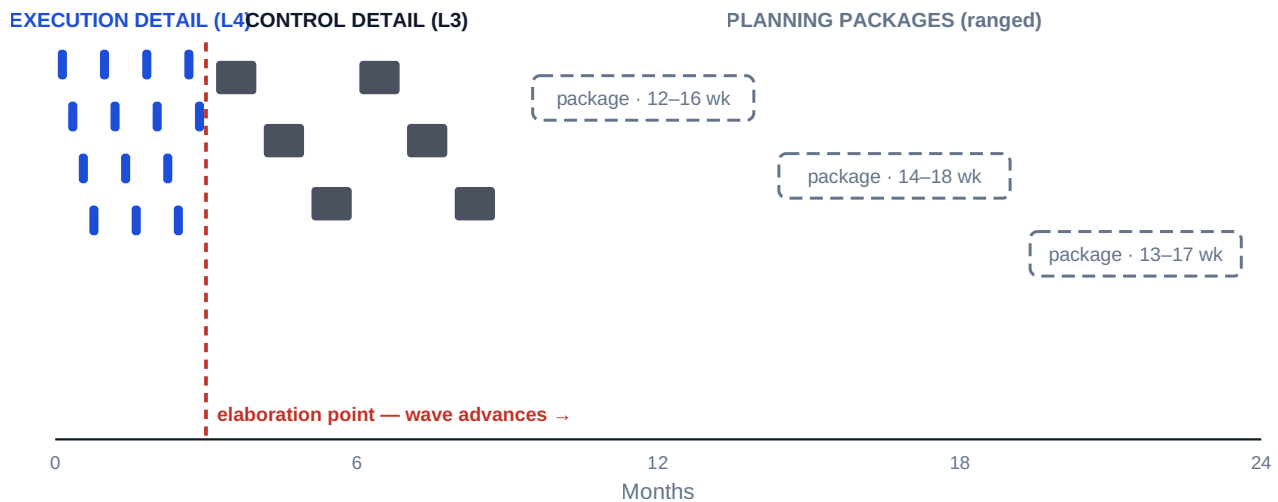


Fig 6.3.2 — The rolling-wave horizon

AI in this KA

Resource optimisation is a genuine AI strength — levelling across thousands of activities and dozens of calendars is combinatorial work machines do better than planners. Governance holds the same shape: the optimiser proposes a levelled plan; the planner verifies the constraint set was real (crew interchangeability, shift rules, site logistics); the leader owns the trade the optimiser cannot see — whether spending float, money or scope is the right currency this month. An optimiser given a wrong constraint produces a beautifully efficient fiction.

■ KEY TERMS — KA 6.3

Term	Meaning
Resource histogram	Demand per period from the loaded schedule.
Smoothing / levelling	Flattening demand within float / accepting a later date under a hard cap.
Constraint	An externally imposed date; modelled as logic, not as a pin.
Milestone	Zero-duration event with an owner and a done-definition; its date is computed.
Rolling wave	Near-term detail, far-term packages, elaborated under change control.
Planning package	A far-wave summary activity with ranged duration awaiting elaboration.

■ SAMPLE MCQS — KA 6.3

MCQ 6.3-A [6.3.1 · Application] A planner clears a resource peak by delaying an activity with $TF = 1$, $FF = 0$ by one week. The immediate consequences are: - A. no schedule effect of any kind - B. project delay of one week - C. the end date holds, the activity's path float is exhausted, and at least one successor's earliest start moves - D. the resource peak worsens

Rationale: One week inside TF protects the end date (not B), but zero FF moves a successor and the path's buffer is now spent (not A) — the smoothing was legal but not free.

MCQ 6.3-B [6.3.3 · Analysis] A 30-month programme shows daily-level tasks throughout, including month 30. The strongest inference is: - A. the planning team is unusually diligent - B. far-horizon detail is manufactured precision that will churn on every update, obscuring real variance - C. the schedule will need no re-baselining - D. rolling-wave planning has been correctly applied

Rationale: Detail beyond the knowable horizon creates update churn and false confidence — the opposite of diligence in effect (A) and the opposite of rolling wave (D); constant churn makes re-baselining more likely, not less (C).

MCQ 6.3-C [6.3.1 · Application] A hard 3-crew cap forces either a 4-week project extension (delay cost USD 45,000/week) or a second-shift waiver at USD 20,000/week for the same 4 weeks (end date held). The value-maximising choice and its saving are: - A. extend: saves USD 100,000 - B. second shift: saves USD 100,000 - C. second shift: saves USD 25,000 - D. they cost the same

Rationale: Extension costs $4 \times 45,000 = 180,000$; the shift premium $4 \times 20,000 = 80,000$ — choosing the shift saves USD 100,000 and keeps the date. A picks the dearer plan; C compares one week only; D ignores the 65,000-per-week difference... which compounds four times.

MCQ 6.3-D [6.3.3 · Recall] A legitimate planning package in a rolling-wave schedule must carry: - A. daily-level task detail for its whole span - B. a ranged duration consistent with its estimate class and a dated elaboration event under change control - C. zero float, to keep pressure on the team - D. a pinned finish date agreed with the sponsor

Rationale: Far-wave honesty is ranged duration plus a governed elaboration point. A is the manufactured precision rolling wave exists to avoid; C fabricates criticality; D is the typed milestone anti-pattern (Case study B).

■ SELF-CHECK — KA 6.3

1. Why is smoothing "spending the risk buffer"? — It consumes float, and float is the project's absorption capacity for the unknown (KA 6.2.2).
2. What makes a milestone honest? — Computed date, named owner, unambiguous done-definition, traceable logic.
3. What must a planning package carry to be legitimate? — A ranged duration consistent with its estimate class and a dated elaboration event under change control.

KNOWLEDGE AREA 6.4

Delivery flow: predictive, agile and hybrid; recovery and forecasting

Topics: 6.4.1 flow across delivery modes · 6.4.2 schedule compression and recovery · 6.4.3 forecasting and scenario analysis.

6.4.1 Flow across delivery modes

Predictive scheduling (this domain so far) plans the whole network and measures variance against baseline. **Agile** delivery (Domain 13 in full) fixes cadence and capacity and lets scope flow: the schedule questions become throughput (items per iteration), cycle time and forecast by velocity. **Hybrid** programmes — the commonest reality — run engineered streams predictively and product streams by cadence, joined at **integration milestones** that appear in the CPM network as fixed-

capacity deliveries. The leader's job is translation: a velocity forecast ("the reporting module completes in 4–6 sprints") enters the network as a ranged duration on the integration activity, and the network returns what the programme needs from the agile stream — the latest acceptable delivery, read from **LS**. Neither mode is senior; the network prices time, the cadence protects focus, and governance (Domain 3, KA 3.1.3) holds both to evidence.

6.4.2 Schedule compression and recovery

The two levers. **Crashing** buys duration with money (more resources, overtime, expediting); **fast-tracking** buys it with risk (overlapping activities that logic preferred in sequence). Both obey the same law: work the critical path, and re-run the passes after every move, because **the critical path migrates**.

WORKED EXAMPLE

Worked example 6.4.2 — the crash that stopped paying.

1. **Setup.** Auriga's client offers a bonus of USD 45,000 per week saved. Expediting C costs USD 30,000 per week, up to two weeks. Should the leader buy one week, two, or none?
2. **Formula.** Crash only while (weeks actually saved × value per week) > crash cost — checked against the re-run passes, not the original network.
3. **Substitution.** Crash C by 1 (8→7): duration 25→24; net gain $45,000 - 30,000 = +15,000$. Crash C by 2 (8→6): path B-D-E-F (2+6+7+5+4) is now co-critical at 24 — duration stays 24; second week's net $0 - 30,000 = -30,000$.
4. **Result.** **Crash one week only:** +USD 15,000. The second week is pure loss.
5. **Interpretation.** Compression is subject to sharply diminishing returns because every crash promotes the next-longest path. The disciplined sequence: identify the binding path → price the cheapest week on it → re-run the passes → repeat until the next week costs more than it is worth. Leaders who mandate "crash procurement by two weeks" from the original network pay for weeks that no longer exist.

Recovery. When the network slips (self-check 6.2.3: D at 10 weeks → 27), recovery options rank by cost-of-time: re-sequencing and logic re-choice (cheapest — KA 6.1's chosen logic); float harvesting from non-critical paths (G's 8 weeks fund nothing on the new critical path, but its resources might); crashing the new critical path (now through D); fast-tracking E against D's tail with an explicit rework risk (Domain 8's risk register prices it); and scope or acceptance re-negotiation (Domain 5) as the honest last resort. A **recovery plan** states the target date, the moves, their costs and risks, and the decision authority spending them — the template is toolkit 6.T.2.

6.4.3 Forecasting and scenario analysis

Three-point estimates. Uncertain durations are ranges, not points. The PERT expected duration and spread for an activity estimated optimistic *o*, most-likely *m*, pessimistic *p*:

$$t_e = (o + 4m + p) / 6 \qquad \sigma = (p - o) / 6$$

WORKED EXAMPLE**Worked example 6.4.3 — Auriga's installation risk.**

1. **Setup.** Installation E is estimated $o = 4$, $m = 5$, $p = 12$ weeks (the long tail: a legacy-system integration problem discovered late).
2. **Formula.** $t_e = (o + 4m + p)/6$; $\sigma = (p - o)/6$.
3. **Substitution.** $t_e = (4 + 20 + 12)/6 = 36/6$; $\sigma = (12 - 4)/6 = 8/6$.
4. **Result.** $t_e = 6.0$ weeks (not 5); $\sigma = 1.33$ weeks.
5. **Interpretation.** The deterministic network used 5 weeks; the risk-weighted expectation is 6 — the skewed tail alone adds a week to the honest forecast, pushing expected completion toward 26 weeks before any risk event "happens". Deterministic dates are the mode, not the mean; boards deserve the mean and the spread. Full quantitative schedule risk analysis — correlating activities, simulating paths, criticality indices — is Domain 8 (KA 8.2), built on exactly this input.

Scenario analysis. Between the single date and the full simulation sits the leader's workhorse: three coherent scenarios (base / threat / opportunity), each a complete re-run of the passes under stated assumptions, each with owners for the assumptions that differ. Scenarios are cheap, auditable, and — because each is a real network — they expose path migration that a percentage-confidence number hides.

The earned-schedule bridge. Once cost joins schedule (Domain 7), progress data yields a time-based forecast: **earned schedule ES** asks when the value now earned was planned to have been earned, and $SPI(t) = ES / AT$ (actual time) measures schedule efficiency in time units. If Auriga's week-22 status shows work that the baseline expected by week 20, $SPI(t) = 20/22 = 0.91$ — the programme is running at 91 % of planned tempo, a signal the currency-based **SPI** famously loses late in a project. Domain 7 (KA 7.3) builds the full machinery; it is flagged here because schedule forecasting belongs to whoever owns the network, not only to the cost engineer.

AI in this KA

Machine forecasting earns trust in exactly one way: calibration. A model that ingests progress data and produces completion forecasts must show its record — forecast vs actual across past periods — before its numbers enter a board pack (Domain 14, KA 14.3's verification workflow). The leader's questions are the auditor's: what data trained it, what does it do when the data thins, who re-ran its arithmetic, and what would make it wrong? Fluent confidence without a calibration record is the machine version of the site manager who is "sure it'll be fine".

■ KEY TERMS — KA 6.4

Term	Meaning
Crashing / fast-tracking	Buying duration with money / with overlap risk.
Path migration	The critical path moving as durations change; why passes are re-run.
Recovery plan	Target date, ranked moves, costs, risks, decision authority.
Three-point / PERT estimate	$t_e = (o+4m+p)/6$, $\sigma = (p-o)/6$.
Scenario analysis	Complete network re-runs under coherent stated assumptions.
Integration milestone	The CPM node where an agile stream's delivery enters the network.

■ SAMPLE MCQS — KA 6.4

MCQ 6.4-A [6.4.2 · Application] Saving a week is worth USD 45,000; crashing the critical activity costs USD 30,000 per week for up to two weeks; after one week of crashing, a parallel path becomes co-critical. The value-maximising decision is: - A. crash two weeks (net +USD 30,000) - B. crash one week (net +USD 15,000) - C. crash nothing (avoid all cost) - D. crash two weeks and fast-track the parallel path at no cost

Rationale: Week one converts to a project week: +15,000. Week two is absorbed by the co-critical path: -30,000. A prices weeks that don't materialise; C leaves +15,000 unclaimed; D invents a free fast-track — overlap always carries rework risk.

MCQ 6.4-B [6.4.3 · Application] An activity is estimated $o = 4$, $m = 5$, $p = 12$ weeks. Its PERT expected duration is: - A. 5.0 weeks - B. 7.0 weeks - C. 6.0 weeks - D. 6.5 weeks

Rationale: $(4 + 4 \times 5 + 12)/6 = 36/6 = 6.0$. A is the most-likely value (the mode); B is the unweighted mean of the three points; D miscounts the weighting.

MCQ 6.4-C [6.4.1 · Analysis] A hybrid programme needs a software module from an agile team for an integration milestone. The schedule-sound way to join the two worlds is: - A. impose the CPM date on the team as a sprint deadline - B. enter the team's velocity-based forecast as a ranged duration and read the latest acceptable delivery from the backward pass - C. exclude the module from the network since agile work cannot be scheduled - D. convert the agile team to predictive planning for the integration period

Rationale: B translates in both directions — evidence-based forecast in, latest-start requirement out. A dictates without evidence; C leaves the network blind at a convergence point; D destroys the team's delivery system to decorate the network.

MCQ 6.4-D [6.4.3 · Application] At week 22, a programme has earned the value its baseline planned to earn by week 20. Its time-based schedule performance index $SPI(t)$ is: - A. 1.10 - B. 0.91 - C. 0.80 - D. 20.0

Rationale: $SPI(t) = ES/AT = 20/22 = 0.91$ — the programme delivers at 91 % of planned tempo. A inverts the ratio; C subtracts the two weeks from the wrong base; D reports earned schedule itself, not the index.

MCQ 6.4-E [6.4.3 · Analysis] Compared with quoting "78 % confidence of the date", giving the board three fully re-run schedule scenarios is stronger because: - A. three numbers always beat one - B. each scenario is an auditable network with named assumptions, showing how the date moves and where the path migrates - C. percentages are always statistically invalid - D. scenarios eliminate the need for risk analysis

Rationale: The scenario's power is auditability and mechanism — assumptions with owners, visible path migration. A is numerology; C overclaims (a calibrated percentage is legitimate, Domain 8); D reverses the relationship — scenarios are inputs to risk analysis, not substitutes.

■ SELF-CHECK — KA 6.4

1. Why does compression show diminishing returns? — Each crash promotes the next-longest path; saved weeks stop converting to project weeks (path migration).
2. What makes a scenario more useful to a board than a confidence percentage? — It is a complete, auditable network with named assumptions — it shows how the date moves, not just that it might.
3. When may a machine forecast enter a board pack? — With a calibration record, verified arithmetic, stated data lineage and a named human owner.

— Advanced topics — Domain 6

6.A.1 The drum: constraint-aware flow

Where one resource is the system's constraint (a single commissioning team, one test rig), throughput scheduling subordinates everything to that drum: buffer it, never starve it, and let non-constraint efficiency go. This reframes utilisation worship — a 100 %-busy non-constraint builds inventory, not progress. In schedule terms: protect the constraint's feed path with deliberate float placement rather than spreading buffer evenly.

6.A.2 Schedule baselines and the update cycle

The control schedule lives on a cycle: status (actuals and remaining durations in), re-run passes, variance vs baseline, forecast, and — rarely, formally — re-baseline under Domain 4's change control. Two integrity rules carried from the family's controls tradition: never edit history (actualised dates are records), and never re-baseline to hide variance — a re-baseline is a governance decision with an audit trail, not a cosmetic one.

6.A.3 PDM edge cases: float under SS and FF links

Precedence-diagram (PDM) links change how float behaves, and two edge cases bite reviewers. **(1) The SS-linked short successor.** P (10 weeks) **SS+2** Q (3 weeks): Q can run weeks 2-5, finishing seven weeks before its predecessor — legal, and correct if the dependency truly binds only the starts; but if Q also needs P's output to finish, the network is missing an FF link, and Q's float is fiction. Paired **SS+FF** links express "starts together, finishes together" honestly. **(2) Duration-driven float.** Under an FF link the successor's float depends on its own duration: lengthen Q and its **LS** moves earlier — planners who "add duration for safety" on FF-linked activities silently destroy their float. The audit habit: on every SS/FF link, ask which end of each activity the logic genuinely binds, and whether the unbound end needs its own link. A network is only as honest as its least-considered link type.

6.A.4 Negative float and the honest schedule

A constraint-bound network can compute $TF < 0$: the plan, as constrained, is late already. Negative float is information — the size of the recovery problem — and suppressing it (deleting constraints, shortening durations by fiat) is the scheduling equivalent of cooking the books. The professional response mirrors KA 6.4.2: quantify it, trace the binding path, price the recovery options, and escalate the decision to whoever owns the trade (Domain 3's escalation design).

— Case study — Domain 6: recovering Auriga (utilities / technology)

Situation. End of week 13. Civil and cabling works D — planned at 7 weeks from week 8 — hits contaminated-ground remediation; the discipline lead's honest re-estimate takes D to 10 weeks (finish week 18). The client's outage window for final commissioning opens at week 25 and closing the window costs USD 45,000 per week of delay. The team re-runs the passes.

Analysis. With D = 10: E starts week 18 (D now later than C's week 16), F finishes week 27 — **two weeks into the penalty window**, and the critical path has migrated to **A-B-D-E-F**. Procurement C, critical all project, now carries $TF = 2$. G still holds float.

The recovery decision. Options priced by the team: (1) crash the old critical path — worthless, C is no longer binding; (2) crash D's remediation with a second civil crew — USD 35,000 for one week; (3) fast-track E against D's tail — start installation in the uncontaminated sections at week 16, accepted rework risk assessed at $20\% \times \text{USD } 60,000 = \text{USD } 12,000$ expected cost, one further week saved; (4) do nothing — $2 \times 45,000 = \text{USD } 90,000$ penalty exposure. The leader takes (2) + (3): cost $\approx \text{USD } 47,000$ expected against USD 90,000 — and finishes at week 25 with the window intact.

What the domain teaches here. Re-run the passes before spending a cent (the instinctive "crash procurement" would have bought nothing); price recovery in expected cost including risk; harvest overlap where the physics genuinely allows it; and report the new float positions to the client honestly — the schedule survived because the remediation estimate was honest a full five weeks before the window (Domain 11's reporting culture, applied to time).

— Case study B — Domain 6: the milestone that was typed, not computed (public programme)

Situation. A government services programme publishes a go-live milestone eighteen months out, set in a ministerial announcement before the L3 schedule existed. When the network is finally built, the backward pass puts the milestone's required start for user-acceptance testing three weeks before the environment that testing needs can exist — $TF = -3$ from day one.

What happened. For six reporting cycles the programme's L1 view showed the milestone green: the date was pinned, and each slip downstream was absorbed by silently shortening the testing window — the classic pin-and-squeeze. In cycle seven an assurance review (Domain 3, KA 3.3) ran the passes without the pin, surfaced the negative float, and forced the choice the pin had deferred: descope the first release (Domain 5), or move the date. The minister moved the date — at a political cost several times what a computed date would have cost eighteen months earlier.

What the domain teaches here. A typed milestone is a promise without a plan. Negative float is not embarrassment to be formatted away; it is the earliest, cheapest warning the programme will ever get (6.A.4). And the leader's protection is procedural, not heroic: milestones enter public commitments only after the backward pass says they exist.

— Executive perspective — Domain 6

What a project leader cannot delegate in this domain:

- **The logic sign-off.** Discipline leads own their links; the leader owns the assembled claim that this network is how the project actually works.
- **Float policy.** Who may spend float, in what increments, reported how — decided before the first progress update, not during the first crisis.
- **The honesty settings.** No pins doing logic's work; negative float reported the cycle it appears; re-baselining only through governance. Schedules fail morally before they fail mathematically.
- **The compression chequebook.** Crash and fast-track decisions are investments under uncertainty — the leader signs the expected-value arithmetic (6.4.2) and the risk acceptance (Domain 8), never a bare instruction to "go faster".
- **The translation duty.** Boards hear dates; teams live in passes, floats and scenarios. The leader is the honest converter between the two languages — both directions.

— Calculation exercises — Domain 6

Exercise 6.1 Using the Auriga network, compute **ES/EF** for every activity and the project duration, showing the convergence at E. Solution. A 0-2 · B 2-8 · C 8-16 · D 8-15 · E $\max(16,15)=16$ -21 · F 21-25 · G 15-17. Duration **25 weeks**; E's start is set by C (16), not D (15) — the convergence rule takes the later predecessor. Common error: taking the earlier predecessor at a merge (giving E 15-20 and a false 24-week project).

Exercise 6.2 Complete the backward pass and state **TF** and **FF** for D and G. Solution. From **LF** = 25: F 21/25 · E 16/21 · C 8/16 · G 23/25 · D **LF** = $\min(E \text{ LS } 16, G \text{ LS } 23) = 16$, **LS** = 9. **D: TF = 1, FF = $\min(16,15) - 15 = 0$. G: TF = 8, FF = 25 - 17 = 8**. Common error: computing D's **FF** against only E (giving 1) — free float tests every successor.

Exercise 6.3 The client offers USD 45,000 per week saved; crashing C costs USD 30,000/week (max 2 weeks); crashing E costs USD 55,000/week (max 1 week). Find the value-maximising plan. Solution. Crash C by 1: 25→24, +15,000. Second week of C: co-critical B-D-E-F holds 24 — -30,000, reject. Crash E by 1 (E is on both paths): 24→23, **45,000 - 55,000 = -10,000**, reject. **Plan: crash C by one week; net +USD 15,000**. Common error: crashing E first because it is "on more paths" — path count doesn't change its negative unit economics.

Exercise 6.4 Activity durations: o = 6, m = 8, p = 16 weeks. Compute **t_e** and **σ**, and state the deterministic bias. Solution. **t_e = (6 + 32 + 16)/6 = 9.0 weeks**; **σ = (16 - 6)/6 = 1.67 weeks**. The deterministic plan at m = 8 understates the expectation by a full week — right-skewed tails always pull the mean above the mode.

Exercise 6.5 A programme's network shows **TF** = -2 on the binding path. List, in cost order, the recovery families and the governance step each requires. Solution. (1) Logic re-choice/re-sequencing — planner's proposal, leader's sign-off; (2) float and resource harvesting from non-binding paths — float-policy authority; (3) crashing the binding path — expected-value case, budget authority; (4) fast-tracking — risk acceptance recorded in the register; (5) scope/acceptance renegotiation — sponsor and change control. Reporting the -2 honestly precedes all five (6.A.4).

— Practitioner's toolkit — Domain 6

Adoption-ready artefacts; adapt headings to your organisation, then keep them stable.

Toolkit 6.T.1 — Schedule quality gate (run before trusting any network)

- [] Zero dangles; zero loops; every lag has a stated reason.
- [] Date constraints listed, each justified as genuine external physics — none doing logic's job.
- [] Durations owned by the executing discipline, estimate class stated (far waves: ranged).
- [] Passes re-run after every change; critical-path durations sum exactly to project duration.
- [] Near-critical paths (within 10 %) listed with their floats.
- [] Float report distinguishes **TF** and **FF**; float policy names who may spend it.
- [] Milestones computed, owned, done-defined; no typed dates.

- [] AI-drafted logic or forecasts marked, verified, and owned by a named human.

Toolkit 6.T.2 — Recovery plan (one page)

Slip statement (activity, cause, size, date detected) · re-run pass results (new duration, new critical path, float table) · options priced in expected cost (re-sequence / harvest / crash / fast-track / renegotiate), each with risk and owner · selected plan and its decision authority · revised commitments and the stakeholder notice list · review date.

Toolkit 6.T.3 — Milestone register

Per milestone: computed date and current **TF** · owner · done-definition (acceptance reference) · contractual status and counterparty · L3 logic trace · reporting treatment (internal / contractual / public) · change history.

— Exam preparation — Domain 6

The calculation traps. Taking the earlier predecessor at a convergence (Exercise 6.1) · computing **FF** against one successor instead of all (Exercise 6.2) · quoting **TF** after it has been spent · crashing from the original network instead of re-running passes (path migration, 6.4.2) · treating the PERT mode as the mean (6.4.3) · reading a pinned milestone as a forecast · confusing project-level float with activity-level slack in reports.

Reflection questions. 1. Your programme's board pack shows every milestone green. What three questions establish whether that is information or formatting? (Computed or typed? Passes re-run this cycle? Where is the float table?) 2. A subcontractor asks for "the float" on their package. What must you settle before answering? (Which float — **TF** or **FF**; the float policy; contractual float ownership — Domain 10.) 3. When did you last see money spent on compression that the passes would have shown was worthless — and which governance step was missing? (6.4.2; toolkit 6.T.2.)

— Domain 6 summary

This domain turned "when will it finish?" from an opinion into an instrument. Planning levels give each audience a true view of one logic; dependency discipline makes the network a model rather than a drawing; the forward and backward passes yield the dates, the two floats and the critical path — with the master project showing why free float can vanish while total float survives, and why convergence points compound slippage. Resources, constraints, milestones and rolling waves connect the network to the real world's crews, windows and horizons. Delivery flow closes the loop: compression priced by expected value and re-run passes, recovery ranked from re-choice to renegotiation, PERT and scenarios replacing single dates with honest ranges, and agile streams joined to the network through ranged integration milestones. The leadership spine throughout: schedules fail morally before they fail mathematically — computed dates, visible float, reported negative float, and machine output verified by named humans are what make a programme's word worth something.

Index

C

Constraint	12
Control schedule	6
Convergence point	9
Crashing / fast-tracking	16
Critical path	9

D

Dangle	6
Date constraint	6

E

ES EF LS LF	9
-------------------	---

F

Forward / backward pass	9
Free float FF	9
FS / SS / FF / SF	6

I

Integration milestone	16
-----------------------------	----

L

Lag / lead	6
------------------	---

M

Milestone	12
-----------------	----

P

Path migration	16
Planning package	12

R

Recovery plan	16
Resource histogram	12
Rolling wave	12

S

Scenario analysis	16
Schedule hierarchy (L1-L4)	6
Smoothing / levelling	12

T

Three-point / PERT estimate	16
Total float TF	9